

claude-windows / da34752f-c664-4262-9602-f5d4cfcc6db4

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde
xer\da34752f-c664-4262-9602-f5d4cfcc6db4\subagents\workflows\wf_c5f5bca5-d2a\agent-
a4a2668b97f4ee382.jsonl`
- SHA-256: `7171ba9726a2096dd84c69d6991bff242aa38c75e2d10799932122ac7479bfce`
- Source modified: `2026-06-12T10:45:31+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_c5f5bca5-d2a`
- session_id: `da34752f-c664-4262-9602-f5d4cfcc6db4`
```

Transcript

1. user (2026-06-12T10:43:40.363Z)

Repo: C:\Users\avidu\Projects\sports-data-collector (Windows). Review commit fd530b5 on branch feat/prep-annotation-proxy (diff vs master: run `git diff master...feat/prep-annotation-proxy` in that repo, or read the files directly).

Changed files:

- src/core/proxy.py (NEW - probe/encode/verify/hash/manifest engine)
- src/cli/curate.py (new prep_annotation_proxy method + argparse subparser + dispatch)
- tests/test_proxy.py (NEW - 22 offline tests)
- docs/INGESTION_PIPELINE.md (step 0 + 2 FAQ entries)

Purpose (ADR-002 in docs/DECISIONS.md): re-encode a source video into a downscaled "annotation proxy" for an annotation vendor, while preserving the EXACT frame count / fps / timeline so frame N on the proxy == frame N on the original (the vendor's CVAT labels are recomputed as seconds=frame/fps by src/parsers/cvat/rally_cvat.py). Invariants: never change fps, never trim, verify parity after encode (delete proxy on failure), record provenance (source+proxy MD5) in a manifest CSV. Suite is 159 green; a live ffmpeg-8.1.1 E2E smoke passed.

A reviewer claims the following defect. ADVERSARIALLY VERIFY it by reading the actual code/files ? try to REFUTE it. Default to refuted if the scenario cannot actually occur or the behavior is acceptable/intended for this tool's scope.

CLAIM [low] verify_parity raises ZeroDivisionError for avg_fps == 0.0 (reachable via --allow-vfr) (C:\Users\avidu\Projects\sports-data-collector\src\core\proxy.py:210) parse_rate('0/1') returns 0.0, and the CLI's pre-check at curate.py:683 is `if src\_info.avg\_fps is None` ? 0.0 passes it. is_vfr() correctly treats avg_fps==0 as VFR, but with --allow-vfr the flow proceeds to verify_parity, where `abs(proxy.avg\_fps - src.avg\_fps) / src.avg\_fps` divides by zero and the command dies with a traceback (after a full, possibly long encode) instead of a clean parity error. The duration branch already guards this (`if src.avg\_fps else 0.2`); the fps branch doesn't.

Proposed fix: In verify_parity, treat avg_fps in (None, 0) as 'fps unavailable'; and/or change the CLI guard to `if not src\_info.avg\_fps`.

2. user (2026-06-12T10:43:46.517Z)

fd530b5 feat(curate): prep-annotation-proxy - frame-identical vendor handoff re-encode (ADR-002) 4fb2d6f docs: ADR-002 - vendor annotation proxies generated in-repo (curate prep-annotation-proxy) (#21)

a2d08f8 docs(cvat): update detect_issues padding rationale to config-driven values (#20)

```
docs/INGESTION_PIPELINE.md | 29 ++++
src/cli/curate.py           | 147 +++++
src/core/proxy.py           | 242 +++++
tests/test_proxy.py         | 309 +++++
4 files changed, 724 insertions(+), 3 deletions(-)
```

3. user (2026-06-12T10:43:47.511Z)

feat/prep-annotation-proxy

4. user (2026-06-12T10:43:53.853Z)

```
1      """Annotation-proxy generation for vendor handoff (ADR-002).
2
3      Vendors annotate in CVAT, where every rally timestamp is later recomputed from
4      *frame indices* (`seconds = frame / fps` ? see `src/parsers/cvat/rally_cvat.py`).
5      A 4K/59.94 source is too heavy to frame-step, which is what pushed the pilot
6      vendor onto a ~30-frame sampling grid (~0.5 s) and broke the ±0.3 s boundary bar.
7      The fix is to hand the vendor a light **annotation proxy** of the *same timeline*:
8      downscaled and scrub-friendly, but identical in frame count, fps, and duration,
9      so frame N on the proxy IS frame N on the original and returned labels apply to
10     the original unchanged.
11
12     The ADR-002 proxy contract enforced here:
13     - video timestamps pass through untouched (`-fps_mode passthrough`, never
14       ``-r``): same frame count, same fps, no trim, no leading padding;
15     - the proxy is VERIFIED against the source (frame count / fps / duration) and a
16       failed proxy must be deleted, never shared ? a silently-wrong proxy poisons
17       golden labels;
18     - provenance is recorded (source + proxy MD5s) so the proxy?original mapping
19       survives the handoff (the indexer keys videos by file MD5; downstream
20       processing must keep running on the ORIGINAL file).
21     """
22     import csv
23     import hashlib
24     import os
25     import shutil
26     import subprocess
27     from dataclasses import dataclass
28     from typing import Any, Dict, List, Optional
29
30
31     class ProxyError(RuntimeError):
32         """A proxy step failed in a way that must stop the handoff (never degrade)."""
33
34
35     #: Columns of the provenance manifest (one row appended per generated proxy).
36     MANIFEST_FIELDS = [
37         "video_id", "source_path", "source_md5", "proxy_path", "proxy_md5",
38         "fps", "frame_count", "duration_s", "source_resolution", "proxy_resolution",
39         "encoder_settings", "created_at",
40     ]
41
42     # Encode defaults (see ADR-002). Height 1080 not 720 (table-tennis ball
43     # visibility); GOP 15 = a keyframe every ~0.25 s at 59.94 fps for snappy
44     # back-stepping; CRF 19 keeps the ball/shuttle legible.
45     DEFAULT_HEIGHT = 1080
46     DEFAULT_CRF = 19
47     DEFAULT_GOP = 15
48     DEFAULT_PRESET = "medium"
49
50
51     @dataclass
52     class ProbeInfo:
53         """The frame?time identity of one video stream, as ffprobe reports it."""
54         width: Optional[int] = None
55         height: Optional[int] = None
56         avg_fps: Optional[float] = None      # avg_frame_rate (frames / duration)
57         r_fps: Optional[float] = None       # r_frame_rate (the container's base rate)
58         nb_frames: Optional[int] = None     # demuxed packet count == frame count
59         duration: Optional[float] = None    # video stream duration, else container
```

```

60
61 @property
62 def resolution(self) -> str:
63     if self.width and self.height:
64         return f"{self.width}x{self.height}"
65     return "unknown"
66
67
68 def parse_rate(value: Optional[str]) -> Optional[float]:
69     """Parse an ffprobe rational rate ('60000/1001', '30/1') to float fps."""
70     if not value:
71         return None
72     try:
73         if "/" in value:
74             num, den = value.split("/", 1)
75             num_f, den_f = float(num), float(den)
76             if den_f == 0:
77                 return None
78             return num_f / den_f
79         return float(value)
80     except ValueError:
81         return None
82
83
84 def probe_video(path: str) -> ProbeInfo:
85     """Probe the frame?time identity of `path` via ffprobe (loud on failure).
86
87     Uses ``-count_packets`` (demux-only, no decode) so the frame count is exact
88     yet cheap even for a 30-minute file. Unlike the fetch path's best-effort
89     probe, a failure here RAISES: parity verification is the whole point of the
90     proxy flow, so a missing ffprobe must stop it, not degrade it.
91     """
92     if not shutil.which("ffprobe"):
93         raise ProxyError("ffprobe not found on PATH (install FFmpeg)")
94     if not os.path.exists(path):
95         raise ProxyError(f"video not found: {path}")
96     import json as _json
97     try:
98         out = subprocess.run(
99             ["ffprobe", "-v", "error", "-select_streams", "v:0",
100              "-count_packets",
101              "-show_entries",
102              "stream=width,height,avg_frame_rate,r_frame_rate,nb_read_packets,duration"
103              ":format=duration",
104              "-of", "json", path],
105             stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True, check=True,
106             ).stdout
107         doc = _json.loads(out)
108     except (subprocess.SubprocessError, OSError, ValueError) as e:
109         raise ProxyError(f"ffprobe failed on {path}: {e}")
110
111     streams = doc.get("streams") or []
112     if not streams:
113         raise ProxyError(f"no video stream found in {path}")
114     s = streams[0]
115
116     def _f(value) -> Optional[float]:
117         try:
118             return float(value)
119         except (TypeError, ValueError):
120             return None
121
122     def _i(value) -> Optional[int]:
123         try:
124             return int(value)

```

```

125         except (TypeError, ValueError):
126             return None
127
128     # Prefer the video stream's own duration; fall back to the container's
129     # (audio encoder priming can stretch the container a few hundredths).
130     duration = _f(s.get("duration"))
131     if duration is None:
132         duration = _f((doc.get("format") or {}).get("duration"))
133
134     return ProbeInfo(
135         width=_i(s.get("width")),
136         height=_i(s.get("height")),
137         avg_fps=parse_rate(s.get("avg_frame_rate")),
138         r_fps=parse_rate(s.get("r_frame_rate")),
139         nb_frames=_i(s.get("nb_read_packets")),
140         duration=duration,
141     )
142
143
144 def is_vfr(info: ProbeInfo, rel_tol: float = 0.001) -> bool:
145     """True when the stream looks variable-frame-rate (or we cannot tell).
146
147     For CFR camera footage r_frame_rate == avg_frame_rate. A mismatch means the
148     `frame / fps` time recompute is approximate ? conservative callers should
149     require an explicit opt-in before proxying such a source.
150     """
151     if info.avg_fps is None or info.r_fps is None or info.avg_fps == 0:
152         return True
153     return abs(info.r_fps - info.avg_fps) / info.avg_fps > rel_tol
154
155
156 def build_proxy_command(src: str, dst: str, height: int = DEFAULT_HEIGHT,
157                         crf: int = DEFAULT_CRF, gop: int = DEFAULT_GOP,
158                         preset: str = DEFAULT_PRESET) -> List[str]:
159     """The ffmpeg invocation that downscales WITHOUT touching the timeline.
160
161     Invariants (ADR-002): no ``-r`` / fps filter (fps must never change), no
162     trim, ``-fps_mode passthrough`` (ffmpeg >= 5.1) so every source frame maps
163     1:1 onto a proxy frame with its original timestamp. ``min(height, ih)``
164     never upscales. Audio is kept (contact sound is the annotator's best
165     boundary cue).
166     """
167     return [
168         "ffmpeg", "-hide_banner", "-loglevel", "warning", "-stats", "-y",
169         "-i", src,
170         "-map", "0:v:0", "-map", "0:a?",
171         "-vf", f"scale=-2:min({height}\\,ih)",
172         "-c:v", "libx264", "-preset", preset, "-crf", str(crf),
173         "-g", str(gop), "-pix_fmt", "yuv420p",
174         "-fps_mode", "passthrough",
175         "-c:a", "aac", "-b:a", "160k",
176         "-movflags", "+faststart",
177         dst,
178     ]
179
180
181 def run_ffmpeg(cmd: List[str]) -> None:
182     """Run an ffmpeg command, streaming its progress to the console."""
183     if not shutil.which("ffmpeg"):
184         raise ProxyError("ffmpeg not found on PATH (install FFmpeg)")
185     try:
186         # No capture: -stats progress and any warnings go straight to the user.
187         subprocess.run(cmd, check=True)
188     except (subprocess.SubprocessError, OSError) as e:
189         code = getattr(e, "returncode", "?")

```

```

190         raise ProxyError(f"ffmpeg failed (exit {code}) ? proxy not produced")
191
192
193     def verify_parity(src: ProbeInfo, proxy: ProbeInfo) -> List[str]:
194         """Check the proxy carries the source's exact frame?time identity.
195
196         Returns a list of human-readable failures (empty == parity holds). Frame
197         count must match EXACTLY; fps within 0.1%; duration within a small slack
198         (AAC priming can pad the container by a few hundredths of a second) that
199         still catches any real trim.
200         """
201         errors = []
202
203         if src.nb_frames is None or proxy.nb_frames is None:
204             errors.append("frame count unavailable on source and/or proxy ? cannot prove
parity")
205         elif src.nb_frames != proxy.nb_frames:
206             errors.append(f"frame count changed: source {src.nb_frames} -> proxy
{proxy.nb_frames}")
207
208         if src.avg_fps is None or proxy.avg_fps is None:
209             errors.append("fps unavailable on source and/or proxy ? cannot prove parity")
210         elif abs(proxy.avg_fps - src.avg_fps) / src.avg_fps > 0.001:
211             errors.append(f"fps changed: source {src.avg_fps:.3f} -> proxy
{proxy.avg_fps:.3f}")
212
213         if src.duration is None or proxy.duration is None:
214             errors.append("duration unavailable on source and/or proxy ? cannot prove
parity")
215         else:
216             tol = max(2.0 / src.avg_fps, 0.2) if src.avg_fps else 0.2
217             if abs(proxy.duration - src.duration) > tol:
218                 errors.append(f"duration changed: source {src.duration:.3f}s -> "
219                             f"proxy {proxy.duration:.3f}s (tolerance {tol:.3f}s)")
220
221         return errors
222
223
224     def md5_file(path: str, chunk_size: int = 1024 * 1024) -> str:
225         """MD5 of a file's bytes (the identity downstream tooling keys videos by)."""
226         digest = hashlib.md5()
227         with open(path, "rb") as f:
228             for chunk in iter(lambda: f.read(chunk_size), b""):
229                 digest.update(chunk)
230         return digest.hexdigest()
231
232
233     def append_manifest_row(manifest_path: str, row: Dict[str, Any]) -> None:
234         """Append one provenance row, writing the header only on first creation."""
235         parent = os.path.dirname(os.path.abspath(manifest_path))
236         os.makedirs(parent, exist_ok=True)
237         is_new = not os.path.exists(manifest_path) or os.path.getsize(manifest_path) == 0
238         with open(manifest_path, "a", newline="", encoding="utf-8") as f:
239             writer = csv.DictWriter(f, fieldnames=MANIFEST_FIELDS)
240             if is_new:
241                 writer.writeheader()
242             writer.writerow({k: row.get(k, "") for k in MANIFEST_FIELDS})
243

```

5. assistant (2026-06-12T10:43:59.605Z)

Now the CLI flow in curate.py.

6. user (2026-06-12T10:44:00.994Z)

```

diff --git a/src/cli/curate.py b/src/cli/curate.py
index fde4826..a6d76c2 100644
--- a/src/cli/curate.py
+++ b/src/cli/curate.py
@@ -643,6 +643,120 @@ class SportsDataCLI:
     print(f"Skipped (no annotations) : {skipped_empty}")
     print(f"Manifest          : {manifest_path}\n")

+
+ def prep_annotation_proxy(self, args):
+     """Re-encode a video into a frame-identical annotation proxy (ADR-002).
+
+     The proxy is what the annotation vendor receives: downscaled and
+     scrub-friendly (dense keyframes) so CVAT frame-step-1 work is cheap,
+     but with the source's EXACT frame count / fps / timeline, so frame N
+     on the proxy is frame N on the original and returned labels apply to
+     the original unchanged. Parity is verified after the encode; a proxy
+     that fails verification is deleted, never shared. Provenance (source +
+     proxy MD5s) is appended to the proxy manifest ? downstream processing
+     (import-local, export, the indexer) keeps running on the ORIGINAL file.
+     """
+     from src.core import proxy as proxylib
+
+     src = args.video_path
+     if not os.path.exists(src):
+         print(f"Error: video '{src}' not found.")
+         sys.exit(1)
+
+     stem = os.path.basename(src).split('.')[0]
+     # Mirror import-local's video_id derivation so the manifest row and a
+     # later DB registration of the same source agree on the key.
+     video_id = args.video_id or stem.lower().replace(" ", "_")
+     out = args.out or os.path.join("data", "videos", "proxies", f"{stem}_proxy.mp4")
+     if os.path.abspath(out) == os.path.abspath(src):
+         print("Error: --out must differ from --video-path.")
+         sys.exit(1)
+     if os.path.exists(out) and not args.force:
+         print(f"Error: proxy '{out}' already exists; pass --force to overwrite.")
+         sys.exit(1)
+
+     try:
+         src_info = proxylib.probe_video(src)
+     except proxylib.ProxyError as e:
+         print(f"Error: {e}")
+         sys.exit(1)
+     if src_info.avg_fps is None:
+         print(f"Error: could not determine fps of '{src}' ? the frame->time "
+               f"contract needs it.")
+         sys.exit(1)
+     if proxylib.is_vfr(src_info) and not args.allow_vfr:
+         print(f"Error: '{src}' looks variable-frame-rate "
+               f"(r={src_info.r_fps} vs avg={src_info.avg_fps}).")
+         print(" The vendor pipeline recomputes times as frame/fps, which is only "
+               "exact for constant-frame-rate sources. Re-run with --allow-vfr to "
+               "proceed anyway (timestamps stay identical to the original, but "
+               "frame/fps seconds are approximate for BOTH files).")
+         sys.exit(1)
+
+     os.makedirs(os.path.dirname(os.path.abspath(out)), exist_ok=True)
+     cmd = proxylib.build_proxy_command(src, out, height=args.height,
+                                       crf=args.crf, gop=args.gop,
+                                       preset=args.preset)
+     print(f"Encoding proxy ({src_info.resolution} -> height<={args.height}, "
+           f"fps {src_info.avg_fps:g} preserved)...")
+     try:
+         proxylib.run_ffmpeg(cmd)

```

```

+         proxy_info = proxylib.probe_video(out)
+     except proxylib.ProxyError as e:
+         print(f"Error: {e}")
+         sys.exit(1)
+
+     errors = proxylib.verify_parity(src_info, proxy_info)
+     if errors:
+         print("Error: proxy FAILED frame<->time parity verification:")
+         for err in errors:
+             print(f" - {err}")
+         try:
+             os.remove(out)
+             print(f"Deleted the failed proxy '{out}' (a silently-wrong proxy "
+                   f"fpoisons golden labels).")
+         except OSError:
+             print(f"WARNING: could not delete the failed proxy '{out}' ? do NOT "
+                   f"share it with the vendor.")
+         sys.exit(1)
+
+     print("Hashing source + proxy (provenance)...")
+     source_md5 = proxylib.md5_file(src)
+     proxy_md5 = proxylib.md5_file(out)
+
+     from datetime import datetime, timezone
+     proxylib.append_manifest_row(args.manifest, {
+         "video_id": video_id,
+         "source_path": os.path.abspath(src),
+         "source_md5": source_md5,
+         "proxy_path": os.path.abspath(out),
+         "proxy_md5": proxy_md5,
+         "fps": f"{src_info.avg_fps:g}",
+         "frame_count": src_info.nb_frames,
+         "duration_s": f"{src_info.duration:.3f}" if src_info.duration is not None else "",
+         "source_resolution": src_info.resolution,
+         "proxy_resolution": proxy_info.resolution,
+         "encoder_settings": (f"libx264 crf={args.crf} g={args.gop} "
+                              f"preset={args.preset} height<={args.height} "
+                              f"fps_mode=passthrough audio=aac160k"),
+         "created_at": datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ"),
+     })
+
+     src_mb = os.path.getsize(src) / (1024 * 1024)
+     out_mb = os.path.getsize(out) / (1024 * 1024)
+     print("\n=== Annotation proxy ready ===")
+     print(f" Video ID      : {video_id}")
+     print(f" Source        : {src} ({src_info.resolution}, {src_mb:.0f} MB, md5 {source_md5})")
+     print(f" Proxy         : {out} ({proxy_info.resolution}, {out_mb:.0f} MB, md5 {proxy_md5})")
+     print(f" Parity        : VERIFIED ? {src_info.nb_frames} frames @ {src_info.avg_fps:g} "
+           f"fps on both")
+     print(f" Manifest      : {args.manifest}")
+     print("\nNext:")
+     print(" 1. Share the PROXY with the vendor (CVAT at frame step 1; first/last "
+           "box of every rally on the exact contact/dead frame).")
+     print(" 2. On delivery, convert-cvat may use --video-path with either file "
+           "(identical fps/frame count by construction).")
+     print(" 3. import-local / export keep pointing at the ORIGINAL file ? the "
+           "indexer keys videos by its MD5.\n")
+
+     def convert_cvat(self, args):
+         """Convert a vendor CVAT rally export into our canonical per-rally CSV.

```

```

@@ -1017,6 +1131,37 @@ def main():
    fetch_parser.add_argument("--retry-backoff", type=float, default=20.0, metavar="SECONDS",

```

```

        help="Base backoff between retries; grows linearly per attempt
(default: 20)")

+ # Prep-annotation-proxy command ? vendor-handoff re-encode (ADR-002)
+ prep_parser = subparsers.add_parser(
+     "prep-annotation-proxy",
+     help="Re-encode a video into a light, frame-identical annotation proxy for "
+     "vendor handoff (ADR-002): downscaled + scrub-friendly, same frame "
+     "count/fps/timeline, parity-verified, provenance recorded")
+ prep_parser.add_argument("--video-path", required=True,
+     help="The ORIGINAL source video (stays the system-of-record
file)")
+ prep_parser.add_argument("--out",
+     help="Proxy output path (default:
data/videos/proxies/<stem>_proxy.mp4)")
+ prep_parser.add_argument("--video-id",
+     help="Manifest key (default: derived from the filename, like
import-local)")
+ prep_parser.add_argument("--height", type=int, default=1080,
+     help="Max proxy height; never upscales (default: 1080 ? "
+     "720 risks losing the table-tennis ball)")
+ prep_parser.add_argument("--crf", type=int, default=19,
+     help="x264 quality, lower=better (default: 19)")
+ prep_parser.add_argument("--gop", type=int, default=15,
+     help="Max keyframe interval in frames; small = snappy "
+     "frame-stepping (default: 15)")
+ prep_parser.add_argument("--preset", default="medium",
+     help="x264 speed/size preset (default: medium)")
+ prep_parser.add_argument("--manifest", default=os.path.join("data",
"annotation_proxy_manifest.csv"),
+     help="Provenance CSV to append to (default:
data/annotation_proxy_manifest.csv; "
+     "commit it ? it is the proxy<->original mapping of record)")
+ prep_parser.add_argument("--allow-vfr", action="store_true",
+     help="Proceed on a variable-frame-rate source (frame/fps "
+     "seconds are approximate; default: refuse)")
+ prep_parser.add_argument("--force", action="store_true",
+     help="Overwrite an existing proxy file")
+
+ # Convert-cvat command ? normalize a vendor CVAT export into our canonical CSV
+ cvat_parser = subparsers.add_parser(
+     "convert-cvat",
@@ -1077,6 +1222,8 @@ def main():
    cli.curate()
    elif args.command == "import-local":
        cli.import_local(args)
+    elif args.command == "prep-annotation-proxy":
+        cli.prep_annotation_proxy(args)
    elif args.command == "convert-cvat":
        cli.convert_cvat(args)
    elif args.command == "validate-delivery":

```

7. user (2026-06-12T10:44:35.958Z)

```

682:         if src_info.avg_fps is None:
parse_rate("0/1") = 0.0
is_vfr(src) = True
CLI guard (avg_fps is None): False
verify_parity raised ZeroDivisionError: float division by zero

```

8. assistant (2026-06-12T10:44:51.948Z)

Confirmed empirically. Let me check the tests to see whether any test pins the `avg\_fps == 0` behavior in `verify\_parity` (which could indicate intent).

9. user (2026-06-12T10:44:53.576Z)

```
53:    assert parse_rate("30/1") == 30.0
54:    assert abs(parse_rate("60000/1001") - 59.94006) < 1e-4
129:    "r_frame_rate": "60000/1001", "avg_frame_rate": "60000/1001",
```

10. user (2026-06-12T10:45:27.111Z)

Structured output provided successfully

11. assistant (2026-06-12T10:45:31.192Z)

Verification complete ? the claim is real at low severity.